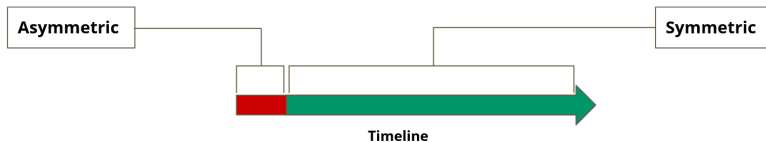


Session 5: Diffie-Hellman

Slides prepared by UMBC Protocol Analysis Lab

Last edited May 28, 2026

How do we actually use encryption?



Number Fields

Number Fields

Let p be a prime, and $g \in \{1, \dots, p-1\}$.

Number Fields

Let p be a prime, and $g \in \{1, \dots, p-1\}$.

Definition

g is called a *generator* for the finite field $\mathbb{Z}/p\mathbb{Z}$ (the numbers mod p) if there exists some f such that

$$f(\{g^1, \dots, g^{(p-1)}\}) = \{1, \dots, p-1\}$$

That is, f is onto, or surjective.

Number Fields

Let p be a prime, and $g \in \{1, \dots, p-1\}$.

Definition

g is called a *generator* for the finite field $\mathbb{Z}/p\mathbb{Z}$ (the numbers mod p) if there exists some f such that

$$f(\{g^1, \dots, g^{(p-1)}\}) = \{1, \dots, p-1\}$$

That is, f is onto, or surjective.

We can represent a generator in CPSA as `(gen)`,
or `(exp (gen) (one))`.

Number Fields

Let p be a prime, and $g \in \{1, \dots, p-1\}$.

Definition

g is called a *generator* for the finite field $\mathbb{Z}/p\mathbb{Z}$ (the numbers mod p) if there exists some f such that

$$f(\{g^1, \dots, g^{(p-1)}\}) = \{1, \dots, p-1\}$$

That is, f is onto, or surjective.

We can represent a generator in CPSA as `(gen)`,
or `(exp (gen) (one))`.

Note that `(gen)` is a term itself in CPSA.

Diffie-Hellman

Alice

Bob

● = Reception ● = Transmission

Diffie-Hellman

Alice

Let Alice have

$$a \in \{1, \dots, p-1\}$$

and g, p

$$\text{Let } A = g^a \bmod p$$

Bob

Let Bob have

$$b \in \{1, \dots, p-1\}$$

and g, p

$$\text{Let } B = g^b \bmod p$$



= Reception



= Transmission

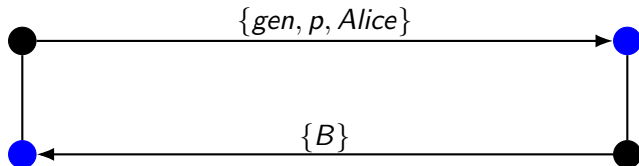
Diffie-Hellman

Alice

Bob

Let Alice have
 $a \in \{1, \dots, p-1\}$
and g, p
Let $A = g^a \bmod p$

Let Bob have
 $b \in \{1, \dots, p-1\}$
and g, p
Let $B = g^b \bmod p$



● = Reception

● = Transmission

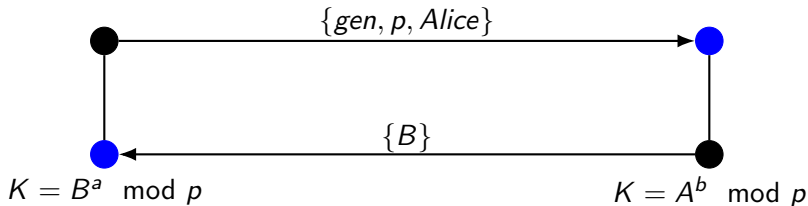
Diffie-Hellman

Alice

Bob

Let Alice have
 $a \in \{1, \dots, p-1\}$
and g, p
Let $A = g^a \mod p$

Let Bob have
 $b \in \{1, \dots, p-1\}$
and g, p
Let $B = g^b \mod p$



$$\begin{aligned} K &= A^b \mod p = (g^a \mod p)^b \\ &= g^{ab} \mod p \\ &= (g^b \mod p)^a = B^a \mod p \end{aligned}$$

● = Reception ● = Transmission

Diffie-Hellman Assumptions

Diffie-Hellman Assumptions

States that its computationally infeasible to compute $g^{(ab)}$ even if g, g^a, g^b are known.

Diffie-Hellman Assumptions

States that its computationally infeasible to compute $g^{(ab)}$ even if g, g^a, g^b are known.

This is how it's possible to exchange keys using Diffie - Hellman securely.

Diffie-Hellman Assumptions

States that its computationally infeasible to compute $g^{(ab)}$ even if g, g^a, g^b are known.

This is how it's possible to exchange keys using Diffie - Hellman securely.

There is a catch:

Diffie-Hellman Assumptions

States that its computationally infeasible to compute $g^{(ab)}$ even if g, g^a, g^b are known.

This is how it's possible to exchange keys using Diffie - Hellman securely.

There is a catch: whether it is actually computationally infeasible is not known, it is an open problem.

Some CPSA hints

- ▶ Need correct origination assumptions
 - ▶ 'a' and 'b' need to be pen-non-orig
 - ▶ msg should be `uniq-orig` (if not it could lead to many shapes)
- ▶ CPSA is used to find structural flaws, not algebraic flaws

Reading Protocol Specifications

What do the keywords mean?

RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)

RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.

RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.

RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.
- ▶ **SHOULD** indicates that something is recommended.

RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.
- ▶ **SHOULD** indicates that something is recommended.
- ▶ **SHOULD NOT** indicates that something is not recommended.

RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.
- ▶ **SHOULD** indicates that something is recommended.
- ▶ **SHOULD NOT** indicates that something is not recommended.
- ▶ **MAY** indicates that something is optional.

What really happens

- ▶ **MUST** actually gets included

What really happens

- ▶ **MUST** actually gets included
- ▶ Nothing else is included ...

Some tips for reading protocol specifications

- ▶ FIND THE SECURITY GOALS
- ▶ Find the messages
 - ▶ “The [party1] relays [data] to [party2]”
 - ▶ “[party2] receives [data]” (figure out who sent it)
 - ▶ “With [data], [party2] does [action]” (how did it get data?)
- ▶ Find what every implementation must consider (*MUST*), especially if you are comparing.
- ▶ Find assumptions in the protocol (DY adversary)
 - ▶ “With private keys kept secret. . .”
 - ▶ “We assume both clients are honest participants”
 - ▶ “A high entropy source is available” (UUID4)

Homework

Modeling from a spec in CPSA

Modeling from a spec in CPSA

Modeling a real world protocol takes some practice, so let's try!

You will have a full week to model SCEP from SecureDNA.

A writeup of the spec can be found in the homework folder.

Please ask for help if you need it, SCEP is complicated and we are here to help!

Survey

Fill in details about the survey here!